Database Query Analysis & Performance Report

وثيقة تحليل واستكشاف اختناقات أداء قاعدة البيانات

مشروع نظام إدارة الطلبات

نوع الوثيقة	تقرير هندسي لتحليل وتحسين الأداء – مشروع التخرج
نقطة النهاية المُحلَّلة	`GET /api/v1/orders/admin` (قائمة الطلبات للمدير)
البيئة التقنية	Laravel (PHP) · MySQL · Docker · k6 · Postman
حجم البيانات	≈ 100,000 سجل في جدول `orders`

52+ → 1-2	ALL → ref	0% → 100%	610 / 610
عدد الاستعلامات	نوع الوصول في EXPLAIN	نجاح اختبار k6	طلبات ناجحة (avg 1.66s)

1. المقدمة وتوصيف المشكلة (Problem Description)

أثناء مرحلة اختبار الحمل والأداء لخدمة الطلبات (`OrderService`) على قاعدة بيانات تضم نحو 100,000 سجل، واجه النظام تحدياً حرجاً تمثّل في استنزاف الذاكرة والبطء الشديد في الاستجابة. وبعد تشخيص قاعدة البيانات عبر سجل الاستعلامات (Query Log) وخطة التنفيذ (`EXPLAIN`)، تبيّن أن السبب الجذري يتكوّن من مشكلتين متضافرتين.

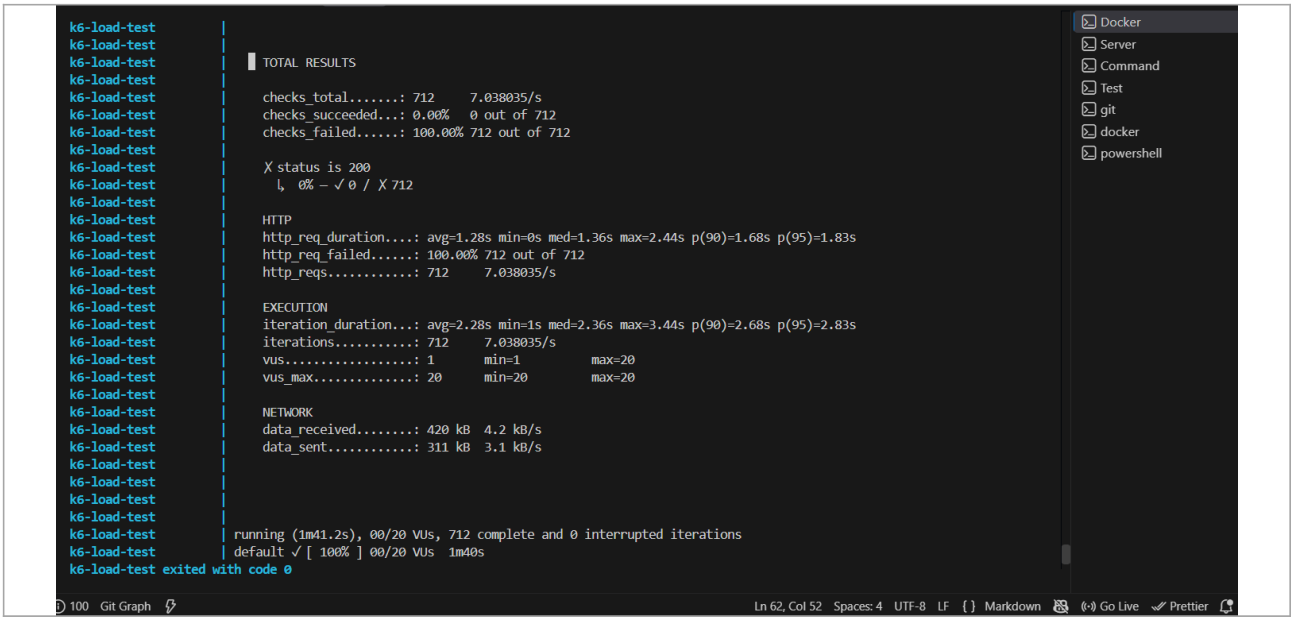
1.1 مشكلة الاستعلامات المتكررة (N+1 Query Problem)

تحدث المشكلة عندما يجلب الكود السجلات الرئيسية (الطلبات – Orders) باستعلام واحد، ثم يمرّ عليها بحلقة تكرارية مباشرة (**Loop**) لجلب بيانات الكيان المرتبط (المستخدم وعناصر الطلب) **لكل طلب على حدة**. في دالة `getAllOrders()` كانت كل دورة تستدعي `findBasicById()` التي تنفّذ استعلاماً مستقلاً على جدول المستخدمين، فيتضاعف عدد الاستعلامات طردياً مع عدد السجلات.

> **ملاحظة منهجية هامة**
>
> تم إجراء اختبارات رصد عدد الاستعلامات (Query Log) واختبارات الـ `EXPLAIN` الأولية على عينة محددة بـ **50 سجلاً فقط** كإثبات مفهوم (Proof of Concept) لضبط النمط ورصد الـ **52 استعلاماً**؛ نظرًا لأن تشغيل الاستعلام غير المحسّن على حجم البيانات الحقيقي في قاعدة البيانات (**100,000 سجل**) كان يؤدي فوراً إلى انهيار خادم PHP واستنزاف الذاكرة (**Memory Exhaustion – Error 500**) تحت الضغط.

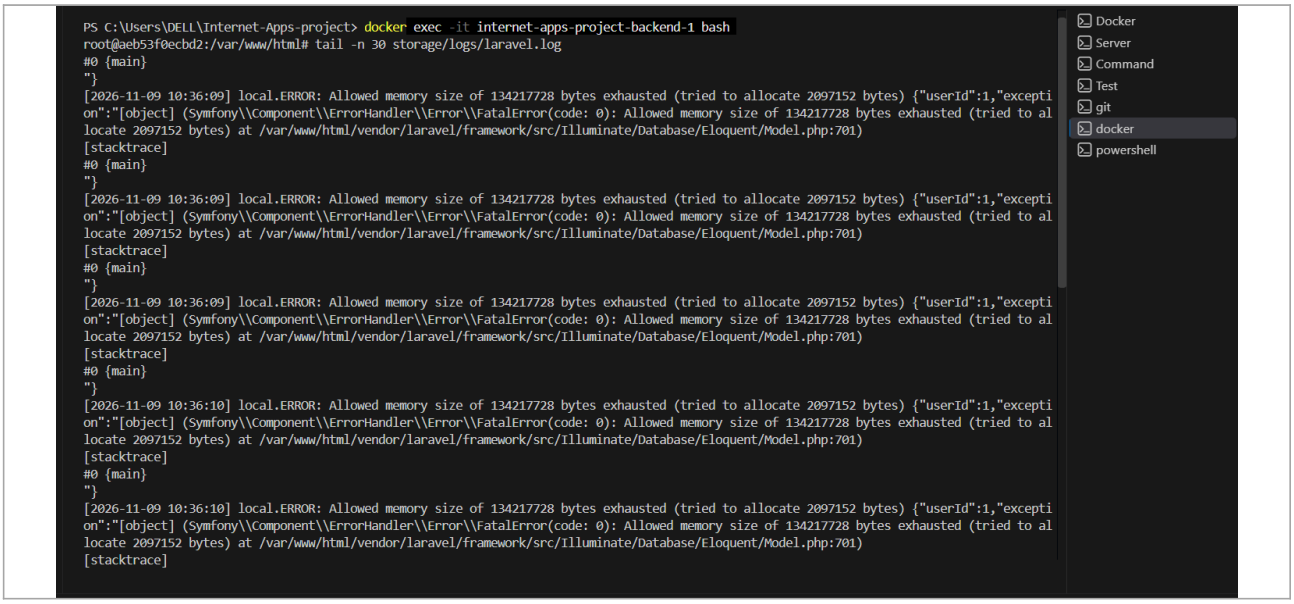
1.2 الدليل العملي: انهيار النظام تحت الضغط

عند تشغيل اختبار الحمل بأداة k6 على النسخة غير المحسّنة (20 مستخدماً افتراضياً) فشلت جميع الطلبات: **712 طلباً فاشلاً من 712 (نسبة النجاح 0%)** كما يظهر في الشكل (1).



الشكل (1): نتيجة اختبار k6 الأولي قبل التحسين – فشل 100% (checks_succeeded = 0.00%) نتيجة استنزاف الذاكرة.

وتؤكد سجلات Laravel الحقيقية (الشكل 2) أن سبب الفشل هو تجاوز حد الذاكرة المسموح (134,217,728 بايت ≈ **128MB**) أثناء تحميل نماذج Eloquent:



الشكل (2): سجلات Laravel Logs تُظهر الخطأ: Allowed memory size of 134217728 bytes exhausted.

1.3 إثبات نمط N+1 عبر سجل الاستعلامات

أظهر سجل الاستعلامات عند جلب عينة الـ 50 سجلاً تنفيذ **52 استعلاماً** منفصلاً في الطلب الواحد: استعلام أساسي واحد لجلب الطلبات + 51 استعلاماً متكرراً لجلب بيانات المستخدمين. ويتكرر الرقم ذاته (`query_count: 52`) في كل طلب، مما يثبت أن النمط ثابت وليس عرَضياً.

```
root@fd2b3b7907fc:/var/www/html# tail -n 20 storage/logs/laravel.log
[2026-11-09 12:22:38] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:39] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:40] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:40] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:40] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:41] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:41] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:42] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:42] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:42] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:43] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:44] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:45] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:45] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:46] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:46] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:46] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:48] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:48] local.INFO: All orders query count. {"query_count":52}
[2026-11-09 12:22:48] local.INFO: All orders query count. {"query_count":52}
root@fd2b3b7907fc:/var/www/html#
```

الشكل (3): سجل الاستعلامات (Query Log) يُثبت نمط N+1 بتكرار 52 استعلاماً لكل طلب.

1.4 تحليل خطة التنفيذ قبل التحسين (EXPLAIN)

باستخدام `tinker` نُفّذ الاستعلام `EXPLAIN SELECT * FROM orders ORDER BY created_at DESC`، وأظهرت النتيجة (الشكل 4) فحصاً كاملاً للجدول دون أي فهرس:

الحقل (Column)	القيمة المرصودة	الدلالة الهندسية
`type`	**ALL**	فحص كامل للجدول (Full Table Scan)
`key`	**NULL**	لا يوجد أي فهرس مستخدم
`rows`	**99,982**	معالجة ما يقارب كامل الجدول في كل طلب
`Extra`	**Using filesort**	فرز ثقيل في الذاكرة لغياب فهرس على `created_at`

```
> dump(DB::select('Explain Select * FROM orders ORDER BY created_at DESC '))

array:1 [
  0 => {#8949
    +"id": 1
    +"select_type": "SIMPLE"
    +"table": "orders"
    +"partitions": null
    +"type": "ALL"
    +"possible_keys": null
    +"key": null
    +"key_len": null
    +"ref": null
    +"rows": 99982
    +"filtered": 100.0
    +"Extra": "Using filesort"
  }
] // vendor/psy/psysh/src/ExecutionLoopClosure.php(55) : eval()'d code:1
= [
    {#8949
      +"id": 1,
      +"select_type": "SIMPLE",
      +"table": "orders",
      +"partitions": null,
      +"type": "ALL",
      +"possible_keys": null,
      +"key": null,
      +"key_len": null,
      +"ref": null,
      +"rows": 99982,
      +"filtered": 100.0,
      +"Extra": "Using filesort",
    },
```

Docker
Server
Command
Test
git
docker
powershell

الشكل (4): EXPLAIN قبل التحسين: type = ALL، rows = 99,982، Extra = Using filesort.

وبتضافر الفحص الشامل مع الفرز بالذاكرة ومع الـ 52+ استعلاماً في الطلب الواحد، استُنزفت ذاكرة PHP بالكامل وانهار الخادم تحت الحمل الفعلي.

2. الحل الهندسي والمعماري (Implemented Optimizations)

لمعالجة الاختناقات وضمان استقرار النظام، طُبّقت ثلاثة تحسينات متكاملة على مستوى طبقة الخدمة (`OrderService`) وقاعدة البيانات:

2.1 تقسيم الصفحات (Pagination)

تم اعتماد **Pagination بواقع 15 سجلاً لكل صفحة**، بحيث تُجلب البيانات على دفعات ثابتة بدلاً من تحميل الجدول كاملاً في الذاكرة دفعة واحدة، فيصبح استهلاك الذاكرة مستقلاً عن حجم الجدول.

2.2 التحميل المُبكر (Eager Loading)

تم تفعيل Eager Loading عبر `with('items')` لجلب العلاقات المرتبطة باستعلام إضافي واحد لكل مجموعة، فينخفض العدد الكلي من **52+** إلى **2–1 استعلام** بغض النظر عن عدد السجلات المعروضة، وبذلك تُقمع مشكلة N+1 نهائياً.

2.3 الفهرس المركب (Compound Index)

أُنشئ فهرس مركب على الحقلين `(user_id, created_at)` في جدول `orders`. يوجّه هذا الفهرس المحرك للوصول المباشر إلى البيانات المرتّبة مسبقاً داخل بنية B-Tree، مما يلغي الفرز العشوائي في الذاكرة (filesort) ويمنع الفحص الشامل للجدول.

```
// OrderService – Pagination + Eager Loading
Order::with('items')->orderByDesc('created_at')->paginate(15);

// Migration – Compound Index
$table->index(['user_id', 'created_at']);
```

مقتطف توضيحي مبسّط لأسلوب التطبيق.

3. جدول المقارنة الشامل (Before vs. After Matrix)

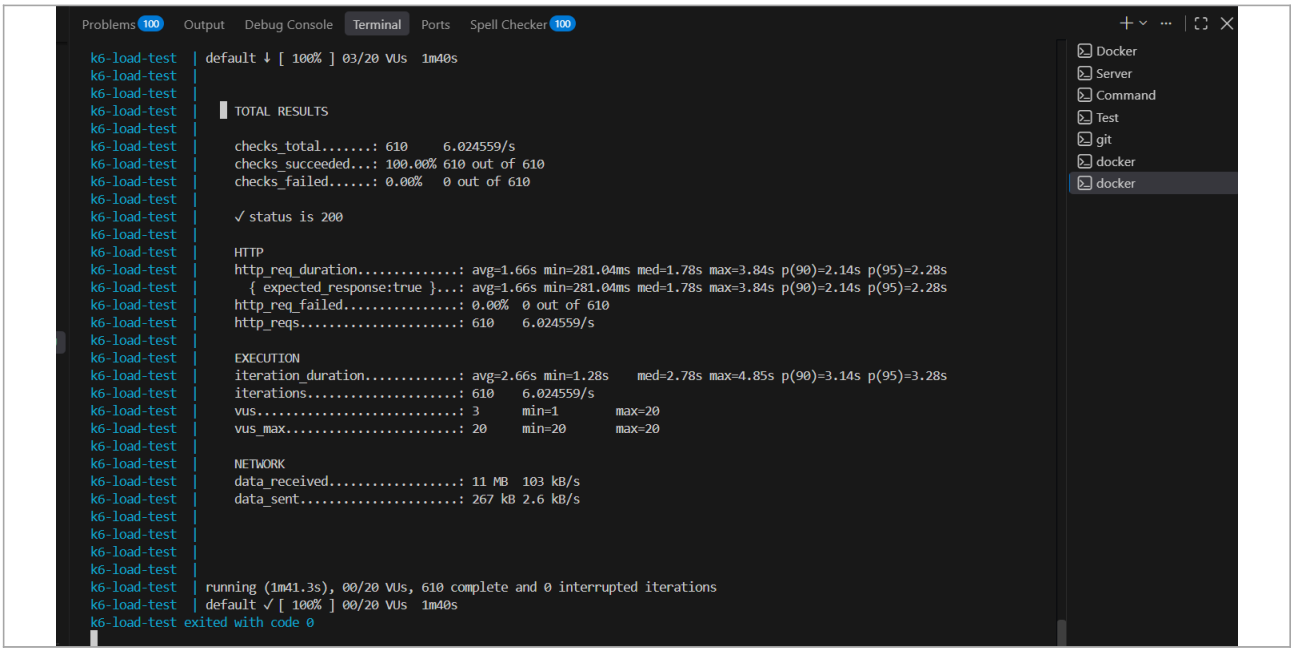
المعيار الهندسي	قبل التحسين (Before)	بعد التحسين (After)	الأثر
نوع الوصول (EXPLAIN type)	**ALL** – فحص كامل (99,982 سطراً)	**ref** – استخدام الفهرس المركب	منع الفحص الشامل
طريقة الترتيب (Extra)	**Using filesort** – فرز ثقيل بالذاكرة	**Backward index scan**	ترتيب جاهز من الفهرس
عدد الاستعلامات (Query Count)	**+52** استعلاماً (يتضاعف مع السجلات)	**2 – 1** استعلام ثابت	القضاء على N+1
استهلاك الذاكرة (Memory)	**استنزاف كامل** للحد (Error 128MB 500)	**مستقر** دون أخطاء استنزاف	اختفاء أخطاء 500
اختبار الضغط k6 (20 VU)	**0%** نجاح (0 من 712 طلباً)	**100%** نجاح (610 من 610 طلباً)	استقرار تحت الحمل

3.1 مؤشرات اختبار k6 بعد التحسين

المؤشر	قبل التحسين	بعد التحسين
نسبة الطلبات الفاشلة (http_req_failed)	(712 من 712) **100%**	(0 من 610) **0.00%**
نجاح الفحوصات (checks)	**0.00%**	**100.00%**
متوسط زمن الطلب (avg)	1.28s (على أخطاء فاشلة)	**1.66s**
الشريحة المئوية p(95)	1.83s (على أخطاء فاشلة)	**2.28s**

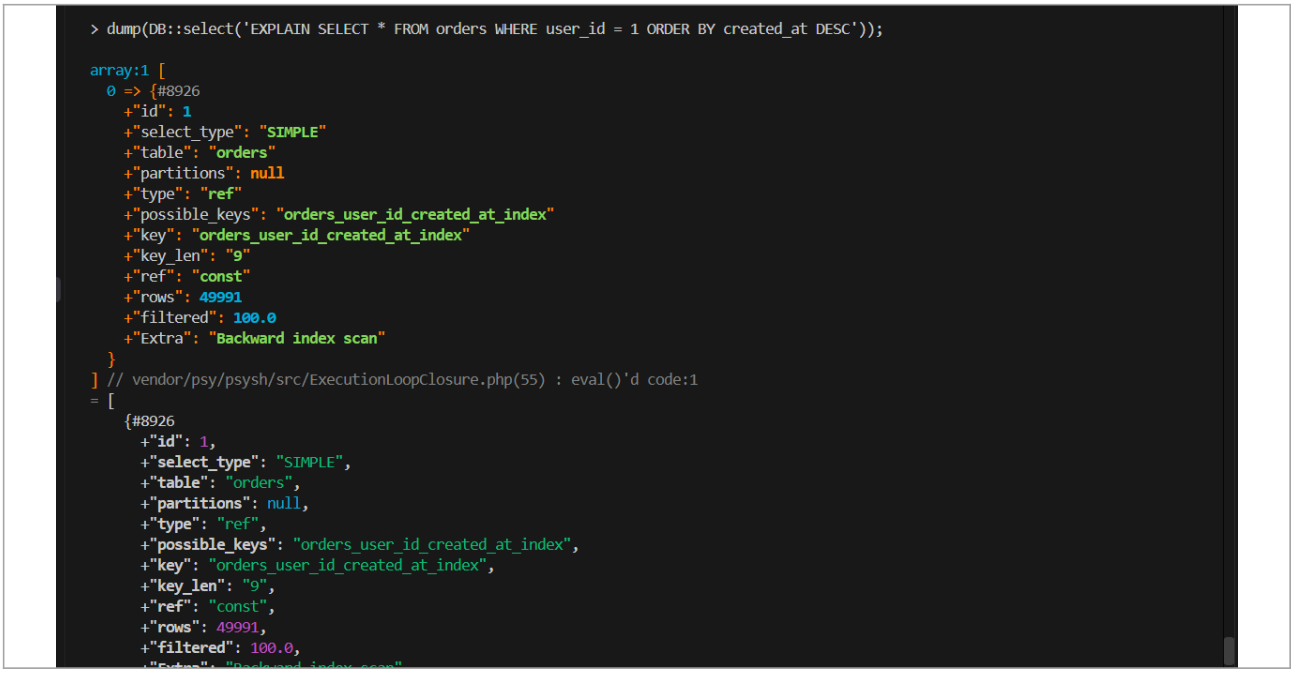
ملاحظة: أزمنة ما قبل التحسين تخص طلبات انتهت بخطأ 500، لذا لا تُعدّ زمن استجابة فعلياً للبيانات؛ أما أزمنة ما بعد التحسين فهي لطلبات مكتملة ببيانات صحيحة.

يوضح الشكل (5) نتيجة اختبار k6 الناجح بعد تطبيق Pagination وEager Loading والفهرسة:



الشكل (5): نتيجة k6 بعد التحسين – نجاح 100% وتنفيذ 610 طلبات بمتوسط زمن 1.66s وبدون أي فشل.

ويُظهر الشكل (6) خطة التنفيذ بعد الفهرسة للاستعلام `EXPLAIN SELECT * FROM orders WHERE user_id = 1 ORDER BY created_at DESC`:



الشكل (6): EXPLAIN بعد التحسين: type = ref، key = orders_user_id_created_at_index، Extra = Backward index scan.

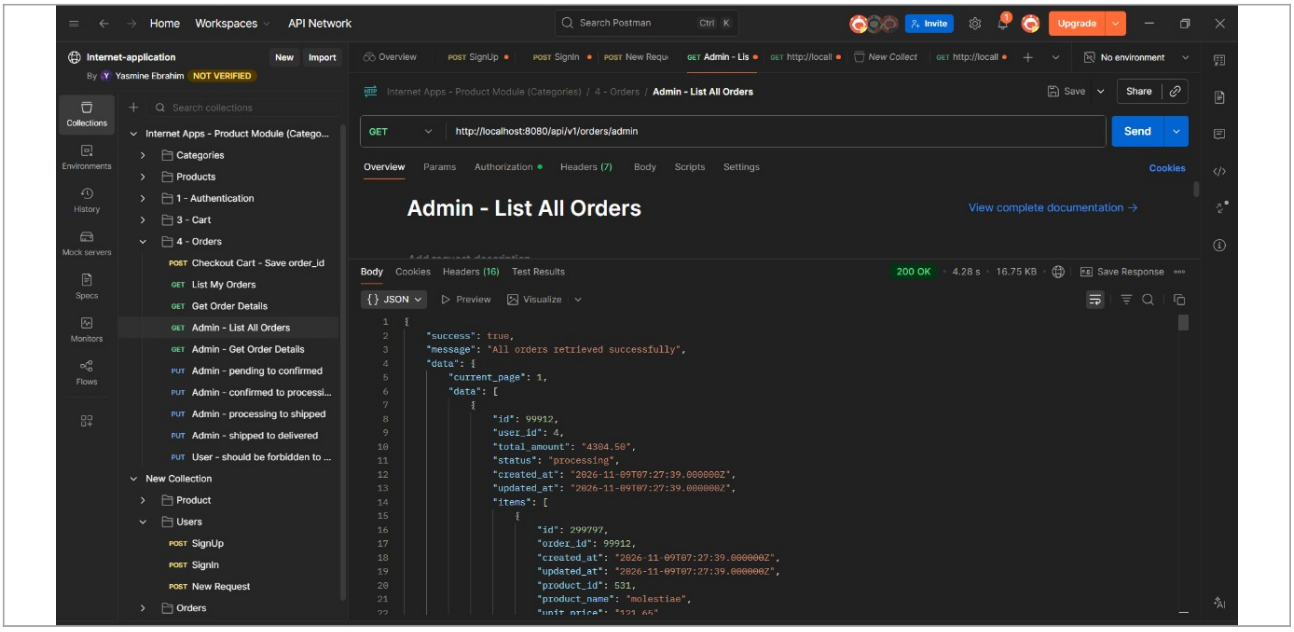
نتيجة التحسين

اختفى `type: ALL` و`Using filesort` تماماً، وأصبح المحرك يستخدم الفهرس `orders_user_id_created_at_index` ويقرأه بالاتجاه العكسي (Backward index scan) لتلبية `ORDER BY created_at DESC` دون أي فرز إضافي.

4. التحقق النهائي (API & Postman Validation)

للتأكد من سلامة الوظيفة بعد التحسينات، جرى اختبار نقطة النهاية المطوّرة `GET /api/v1/orders/admin` عبر Postman، وأكدت النتيجة (الشكل 7) ما يلي:

- حالة الاستجابة: **OK 200** مع `success: true` والرسالة «All orders retrieved successfully».
- عمل الـ Pagination بنجاح: ظهور `current_page: 1` وتقسيم النتائج إلى صفحات بدلاً من إرجاع الجدول كاملاً.
- اكتمال البيانات المرتبطة: عناصر الطلب (`items`) مُحمّلة ضمن كل طلب عبر Eager Loading.



الشكل (7): اختبار Postman لنقطة نهاية المدير – استجابة OK 200 تُظهر نجاح التقسيم وسلامة البيانات.

5. اختبار فصل القراءة والكتابة (Read/Write Verification – k6)

بعد اعتماد بنية قاعدة بيانات بخادمين (`mysql-primary` للكتابة و`mysql-replica` للقراءة)، أُجري اختبار k6 مخصص عبر السكربت `tests/k6/read-write-verification.js` للتحقق من أن الطلب الذي يُنشأ يظهر **مباشرة** في قائمة طلبات الإدارة دون تأخر في النسخ (Replica Lag).

5.1 خطوات الاختبار

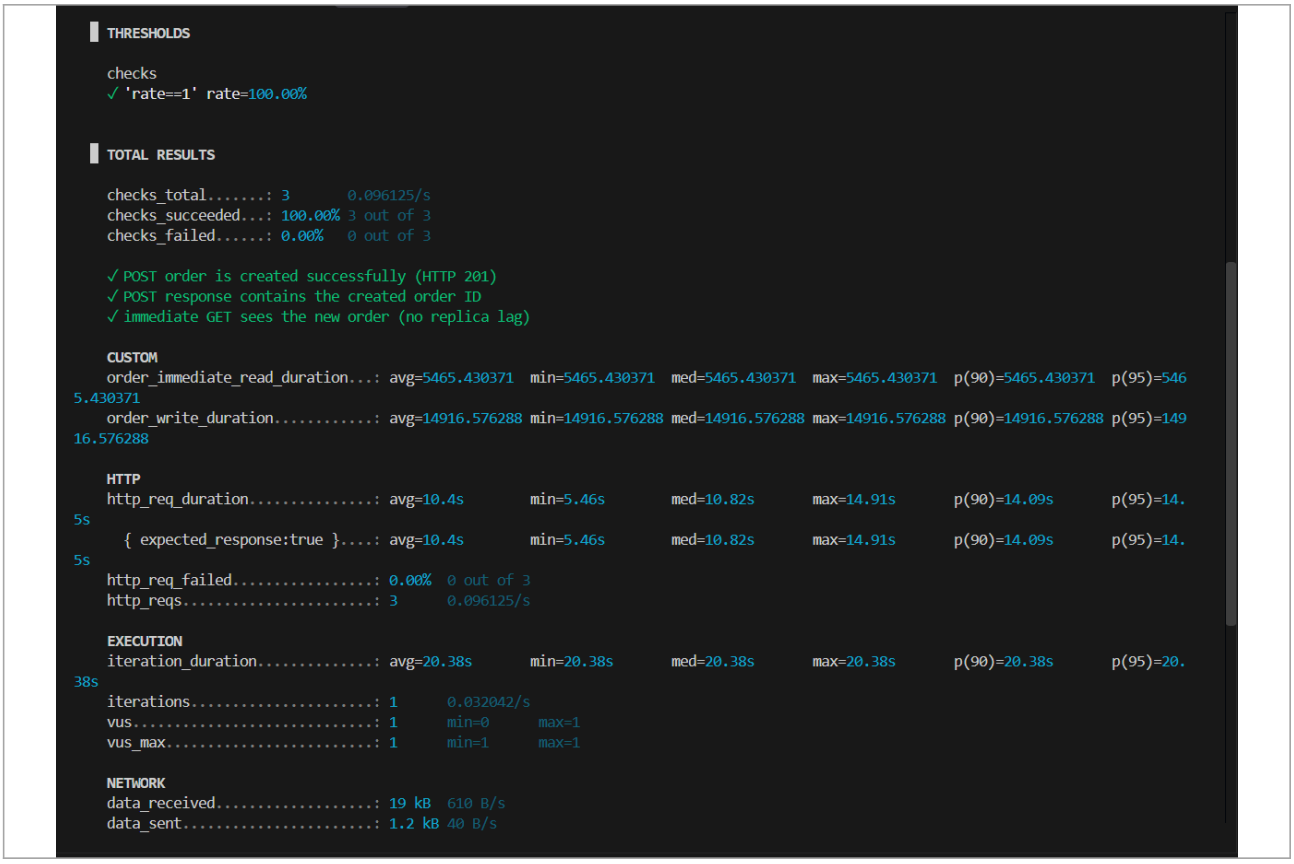
1. تسجيل الدخول عبر `POST /api/v1/users/sign-in` والحصول على رمز JWT.
2. إنشاء طلب بكمية واحدة من منتج محدد عبر `POST /api/v1/orders`.
3. استخراج رقم الطلب من الاستجابة.
4. استدعاء `GET /api/v1/orders/admin` مباشرة والتحقق من ظهور الطلب الجديد.
5. عرض نتائج الفحوص وقياس زمني الكتابة والقراءة.

الإعداد الافتراضي: مستخدم افتراضي واحد وتكرار واحد، وكل تكرار ناجح ينشئ طلباً حقيقياً ويستهلك وحدة من المخزون، لذا لا يُرفع الحمل إلا في بيئة اختبار وبمخزون كافٍ.

5.2 نتائج الاختبار

الفحص / المؤشر	النتيجة	الحالة
إنشاء الطلب بنجاح (HTTP 201)	نجح	✓
الاستجابة تحتوي معرّف الطلب المُنشأ	نجح	✓
GET الفوري يرى الطلب الجديد (no replica lag)	نجح	✓
نسبة نجاح الفحوص (checks)	**100%** (3 من 3)	✓
الطلبات الفاشلة (http_req_failed)	**0.00%** (0 من 3)	✓

زمن الكتابة order_write_duration	≈ 14,917 ms	قياس مرجعي
زمن القراءة الفورية order_immediate_read_duration	≈ 5,465 ms	قياس مرجعي



الشكل (8): نتيجة اختبار k6 للتحقق من الكتابة ثم القراءة الفورية: نجاح الفحوص الثلاثة (100%) وعدم فشل أي طلب.

ملاحظة: الأزمنة أعلاه من تكرار واحد فقط (مستخدم افتراضي واحد)، لذا هي قياس مرجعي وليست متوسطاً إحصائياً، ولم تُحلَّل أسبابها في هذه الوثيقة.

حدود التحقق (Validation Limitations)

نجاح القراءة الفورية **لا يثبت وحده** أن `GET` خُدم من الـ Replica أو أن `POST` نُفّذ على الـ Primary؛ يلزم التحقق من المضيف عبر مراقبة اتصالات واستعلامات قاعدة البيانات.

الإعداد `sticky => true` يضمن توجيه القراءة إلى اتصال الكتابة بعد كتابة ضمن **طلب Laravel نفسه**، وليس تلقائياً بين طلبَي HTTP منفصلين (POST ثم GET). لذلك يثبت الاختبار ظهور الطلب فعلياً بعد الإنشاء، لكنه لا يثبت أن `sticky` هو سبب الاتساق.

6. مقايضات أداء الفهارس (Index Performance Trade-offs)

سؤال التفكير: هل الفهرس مجاني؟ وكيف يؤثر على عمليات القراءة والكتابة؟

الجانب	عمليات القراءة (SELECT / ORDER BY)	عمليات الكتابة (INSERT / UPDATE / DELETE)
التأثير	تسريع كبير	إبطاء طفيف

السبب	شجرة B-Tree مرتّبة تتيح البحث والترتيب دون فحص الجدول أو filesort	كل تعديل يستلزم تحديث الفهرس وإعادة بناء ترتيبه ليبقى متسقاً مع الجدول

الخلاصة: يُسرّع الفهرس عمليات القراءة والترتيب عبر أشجار B-Tree، لكنه يبطئ قليلاً عمليات الكتابة والتعديل لأن كل `INSERT` أو `UPDATE` أو `DELETE` يتطلب إعادة بناء ترتيب الفهرس، كما يستهلك مساحة تخزينية إضافية. ولذلك يُنشأ الفهرس حيث تتركز الاستعلامات المتكررة (كالتصفية بـ `user_id` والترتيب بـ `created_at` في جدول الطلبات الكثير القراءة) ولا يُفرَط في إنشائه على كل الأعمدة.

7. الخلاصة والنتائج

- أُثبت نمط **N+1** (52 استعلاماً لعينة 50 سجلاً) وعُزل الفحص الشامل للجدول (`type: ALL`, `rows: 99,982`).
- عالج الثلاثي **Pagination + Eager Loading + Compound Index** مشكلة الذاكرة والبطء من جذورها.
- تحوّل سلوك النظام تحت الضغط من **فشل كامل (0%)** إلى **نجاح تام (100%)** لـ 610 طلبات متزامنة بمتوسط 1.66s.
- أكد اختبار k6 للقراءة والكتابة ظهور الطلب الجديد فوراً في قائمة الإدارة (3 من 3 فحوص ناجحة)، مع بيان حدود هذا الدليل بخصوص توجيه الـ Replica.
- حافظت نقطة النهاية `GET /api/v1/orders/admin` على سلامة الوظيفة (OK 200) مع بيانات مقسّمة إلى صفحات.